

Raspberry Pi Docker Swarm Setup

In diesem Projekt wird ein Cluster aus mehreren Raspberry Pis mithilfe von Docker Swarm aufgesetzt. Das Ziel ist, eine Pi-Berechnung (Monte-Carlo-Methode) zu verteilen und damit die Möglichkeiten der Parallelisierung zu demonstrieren.

Voraussetzungen

1. **Raspberry Pi** (mindestens 3 Stück empfohlen).
 2. **Betriebssystem**: Debian-/Raspbian-basiertes System.
 3. **Docker-Version**: Auf allen Pis sollte die gleiche Docker-Version installiert sein (getestet mit Docker 27.4.0).
 4. **Funktionierende Netzwerkverbindung** zwischen allen Pis.
-

1. Docker installieren

1.1 Vorhandene Docker-Installationen prüfen

```
which docker
which containerd
ls -al /usr/bin | grep docker
```

Wenn dabei keine relevanten Einträge auftauchen, ist Docker wahrscheinlich nicht installiert.

1.2 System aktualisieren

```
sudo apt-get update
sudo apt-get upgrade -y
sudo reboot
```

(Der Neustart ist wichtig, um sicherzustellen, dass alle Pakete korrekt aktualisiert sind.)

1.3 Docker installieren

1. Abhängigkeiten installieren:

```
sudo apt-get install -y \
    apt-transport-https \
    ca-certificates \
    curl \
    gnupg \
    lsb-release
```

2. Docker GPG-Schlüssel hinzufügen:

```
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg
--dearmor -o /usr/share/keyrings/docker-archive-keyring.gpg
```

3. Docker-Repository hinzufügen:

```
echo \
  "deb [arch=$(dpkg --print-architecture) signed-
  by=/usr/share/keyrings/docker-archive-keyring.gpg]
  https://download.docker.com/linux/debian \
  $(lsb_release -cs) stable" | sudo tee
/etc/apt/sources.list.d/docker.list > /dev/null
```

4. Docker installieren:

```
sudo apt-get update
sudo apt-get install -y docker-ce docker-ce-cli containerd.io
docker-buildx-plugin docker-compose-plugin
```

1.4 Docker einrichten & testen

1. Docker-Dienste aktivieren und starten:

```
sudo systemctl enable docker
sudo systemctl start docker
```

2. Prüfen, ob Docker läuft:

```
sudo systemctl status docker
```

3. Funktionstest mit „Hello World“:

```
sudo docker run hello-world
```

1.5 (Optional) Docker ohne `sudo` nutzen

```
sudo usermod -aG docker $USER
```

Anschließend ab- und wieder anmelden, damit die Gruppenzugehörigkeit aktiv wird.

1.6 Versionen prüfen

```
docker --version  
docker compose version
```

(Bei allen Pis sollten diese Versionen übereinstimmen, z. B. Docker 27.4.0, Docker Compose v2.31.0.)

2. Docker-Image bauen

Da das Image aus keinem Registry gezogen wird, muss es **auf jedem Pi** lokal gebaut werden.

1. Wechsle in das Verzeichnis, das die **Dockerfile** enthält (z. B. **pi_cluster**).
2. Führe folgenden Befehl aus:

```
docker build -t pi_cluster:latest .
```

3. Wiederhole den Vorgang auf **jedem** Raspberry Pi, der am Swarm teilnimmt.

3. Docker Swarm initialisieren

3.1 Swarm auf dem Manager starten

Auf **einem** Pi (Manager-Node) ausführen:

```
docker swarm init
```

- Als Ausgabe erhält man u. a. einen **Join-Token** für die Worker.
- Notiere dir auch die **IP-Adresse** des Manager-Pis.

3.2 Worker dem Swarm hinzufügen

Auf **jedem** Worker-Pi ausführen (Beispiel):

```
docker swarm join --token <TOKEN> <MANAGER-IP>:2377
```

- Ersetze <TOKEN> durch den bei `docker swarm init` ausgegebenen Token.
 - Ersetze <MANAGER-IP> durch die IP des Manager-Pis.
-

4. Docker Stack deployen

1. Stack starten (auf dem Manager-Pi):

```
docker stack deploy -c docker-stack.yaml pi_cluster
```

- `pi_cluster` ist der frei wählbare Name des Stacks.

2. Status prüfen:

```
docker stack ps pi_cluster  
docker service ls
```

5. Nutzung

Sobald der Stack läuft, kann man sich über den interaktiven Modus mit der Shell des main Containers verbinden:

```
docker exec -it <Container-ID des pi_cluster.main> bash
```

innerhalb des Containers kann dann das main Skript gestartet werden mit:

```
python main.py
```

- **Eingabe** einer Zahl (z. B. `100000`) verteilt die Berechnung (Samples) auf alle Worker.
 - `auto` startet den automatisierten Testmodus, welcher nacheinander verschiedene Sample größen durchrechnet.
 - `q` oder `quit` beendet die Main-Anwendung.
-

6. Stack entfernen

Zum Entfernen aller Services und Container:

```
docker stack rm pi_cluster
```

(Warte kurz, bis alle Services beendet sind.)

7. Weiterführende Hinweise

- Die CPU-Temperatur und andere externe Faktoren können die Performance beeinflussen.
- Für noch genauere Tests empfiehlt sich das Monitoring der Raspberry Pis (z. B. mit Grafana, Prometheus oder ähnlichen Tools).